ÉCOLE NORMALE SUPÉRIEURE PARIS-SACLAY

Master's Internship Report

Towards a Next-Generation Univalent Proof Assistant

Raphaël Sterbac
Under the supervision of **Jon Sterling**
(February - July 2026)

UNIVERSITY OF CAMBRIDGE
DEPARTMENT OF
COMPUTER SCIENCE AND TECHNOLOGY

Abstract

1 Introduction

2 Universe hierarchies

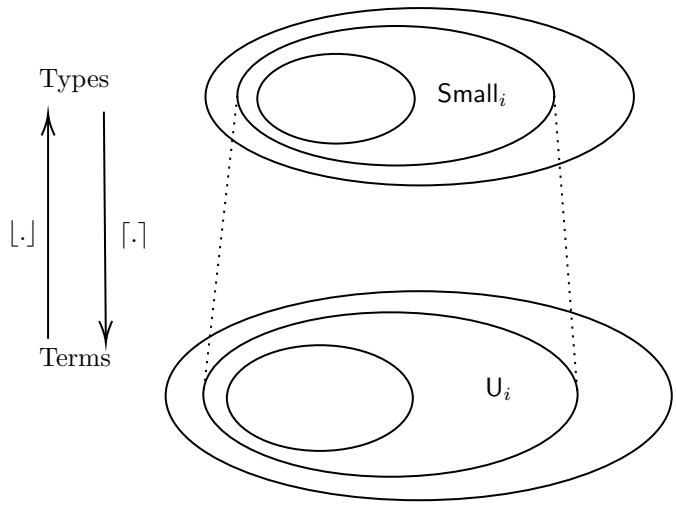

Figure 1: Our universe hierarchy and its reflection

3 A new algebraic Universe Hierarchy

3.1 The syntax

In this section we define a new syntax for universe hierarchies, which provides an algebraic presentation for explicit (open) universes, and can be understood as a variant of universes à la Coquand. The main idea is to introduce a "smallness" judgement, which correspond to the reflection of U_i in the Type world. The rules of this system $\mathcal{T}_F$ are described in the following.

$$\textsc{univ-decoding-small}\quad \frac{\Gamma \vdash A : \mathsf{U}_i}{\Gamma \vdash \lfloor A \rfloor_i\ \mathsf{Smll}_i} \qquad \textsc{univ-coding-formation}\quad \frac{\Gamma \vdash A : \mathsf{Smll}_i}{\Gamma \vdash \lceil A \rceil_i : \mathsf{U}_i} \qquad \textsc{univ-decoding-formation}\quad \frac{\Gamma \vdash A : \mathsf{U}_i}{\Gamma \vdash \lfloor A \rfloor_i\ \mathsf{Type}}$$

$$\textsc{univ-coding-congruence}\quad \frac{\Gamma \vdash A \equiv B \qquad \Gamma \vdash A\ \mathsf{Smll}_i}{\Gamma \vdash \lceil A \rceil_i \equiv \lceil B \rceil_i : \mathsf{U}_i} \qquad \textsc{univ-decoding-congruence}\quad \frac{\Gamma \vdash A \equiv B : U_i}{\Gamma \vdash \lfloor A \rfloor_i \equiv \lfloor B \rfloor_i}$$

In this formulation, we require the rule UNIV-DECODING-INJ explicitely, but we don't ask for the closure under type connectives. The other new rule UNIV-DECODING-IMAGE states that Smll_i is the image of $\lfloor . \rfloor_i$, by requiring explicitely that $\lceil . \rceil_i$ is the right-inverse of $\lfloor . \rfloor_i$.
Note that there is no coherence issue related to this, as we only ask $\lceil . \rceil_i$ and $\lfloor . \rfloor_i$ to be inverses of each other on the Smll_i part of Type.[1]

$$\textsc{univ-decoding-inj}\quad \frac{\Gamma \vdash \lfloor A \rfloor_i \equiv \lfloor B \rfloor_i}{\Gamma \vdash A \equiv B : U_i} \qquad \textsc{univ-decoding-image}\quad \frac{\Gamma \vdash A : \mathsf{Smll}_i}{\Gamma \vdash \lfloor \lceil A \rceil_i \rfloor_i \equiv A}$$

[1] JMS: Didn't quite understand this. The proposed rules are exactly asking these things to be proper inverses (the inj rule is equivalent to an η law that makes the two things mutually inverse).

Then, we need a smallness rule corresponding to every type formation rule.

$$\textsc{dfun-formation}\quad \frac{\Gamma \vdash i+1 \leq j}{\Gamma \vdash \mathsf{U}_i : \mathsf{Smll}_j} \qquad \textsc{dfun-code-formation}\quad \frac{\Gamma \vdash M\ \mathsf{Smll}_i \qquad \Gamma, x : M \vdash N\ \mathsf{Smll}_i}{\Gamma \vdash (x : M)\ \mathsf{to}\ N[x]\ \mathsf{Smll}_i}$$

For the presentation to be algebraic, we axiomatize Smll_i as a proposition $\mathsf{Smll}_i(A) : \mathsf{Prop}$. This is justified by the rule UNIV-DECODING-INJ, and thus $A\ \mathsf{Smll}_i$ will be a notation for $p : \mathsf{Smll}_i(A)$. This notation does not depend on the term p, as Smll_i is a proposition.
The rules of $\mathcal{T}_F$ are enough to recover a coherent universe hierarchy, and thus $\mathcal{T}_F$ is an algebraic presentation for natural universes.

Proposition 3.1.1 (Derivability of codes and lifts)**.** *The following lifting function and codes for type connectives are coherent.*

- $\uparrow_i^j (a) := \lceil \lfloor a \rfloor_i \rceil_j$
- $a\ \mathit{to}_{U_i} b := \lceil (x : \lfloor a \rfloor_i)\ \mathit{to}\ \lfloor b \rfloor_i [x] \rceil_i$
- $U_i^j := \lceil U_i \rceil_j$

Hence, the following rules are derivable.

$$\textsc{dfun-decoding}\quad \frac{\Gamma \vdash a : U_n \qquad \Gamma, x : \lfloor a \rfloor_n \vdash b : U_n}{\Gamma \vdash \lfloor a\ \mathit{to}_{U_n} b \rfloor_n \equiv (x : \lfloor a \rfloor_n)\ \mathit{to}\ \lfloor b \rfloor_n [x]} \qquad \textsc{dfun-lifting}\quad \frac{\Gamma \vdash a : U_n \qquad \Gamma, x : \lfloor a \rfloor_n \vdash b : U_n \qquad \Gamma \vdash n < p}{\Gamma \vdash \uparrow_n^p (a\ \mathit{to}_{U_n} b) \equiv \uparrow_n^p (a)\ \mathit{to}_{U_p} \uparrow_n^p (b))}$$

$$\textsc{univ-code-decoding}\quad \frac{}{\Gamma \vdash \lfloor U_m^n \rfloor_n \equiv U_m} \qquad \textsc{univ-code-lifting}\quad \frac{\Gamma \vdash n < m}{\Gamma \vdash \uparrow_n^p (U_m^n) \equiv U_m^p : U_p} \qquad \textsc{lifting-decoding}\quad \frac{\Gamma \vdash a : U_m \qquad \Gamma \vdash n < m}{\Gamma \vdash \lfloor \uparrow_m^n (a) \rfloor_n \equiv \lfloor a \rfloor_m}$$

$$\textsc{lifting-lifting}\quad \frac{\Gamma \vdash a : U_m \qquad \Gamma \vdash m < n < p}{\Gamma \vdash \uparrow_n^p (\uparrow_m^n (a)) \equiv \uparrow_m^p (a) : U_p}$$

3.2 Equivalence with a strict Tarski-style hierarchy

Conversely, a theory with Tarski universes satisfying all the equations of proposition 3.1.1, denoted $\mathcal{T}_T$, can be shown to have an injective decoding family. This result is not immediate and relies on the normalisation of the Tarski system.

Theorem 3.2.1 (Admissibility of Injectivity)**.** *The following rule is admissible in $\mathcal{T}_T$:*

$$\textsc{univ-decoding-inj}\quad \frac{\Gamma \vdash El_i(a) \equiv El_i(b)}{\Gamma \vdash A \equiv B : U_i}$$

Proof. We can show the normalisation of the theory with an Artin glueing argument. This provides a normalisation by evaluation map $\mathsf{nf} : \mathsf{Type} \to \mathsf{Type}_N$ which is a section of the quotient map $\langle . \rangle : \mathsf{Type}_N \to \mathsf{Type}$, where Type_N is the sort of normal types. Those are of the form

$$M, N = \mathsf{U}_i \mid \Pi(M, N) \mid \mathsf{El}_i(R)$$

where R is neutral. Then, we proceed by induction on the normal forms $a_N = \mathsf{nf}(a)$ and $b_N = \mathsf{nf}(b)$. Notice that we need to prove a stronger statement : if $\mathsf{El}_i(a) = \mathsf{El}_j(b)$ then $i = j$ and $a = b$.

<u>First case:</u> $a_N = \mathsf{U}^i_j$ and $b_N = \mathsf{U}^i_k$. Then, with the equation $\mathsf{El}_i(a_N) = \mathsf{El}_i(b_N)$ we get $\mathsf{U}_k = \mathsf{U}_i$ which implies $i = j$, and we are done : $\mathsf{U}^i_j = \mathsf{U}^i_k$
<u>Second case:</u> $a_N = \Pi^i\ c\ d$ and $b_N = \Pi^i\ c'\ d'$. Then the equation $\mathsf{El}_i(a_N) = \mathsf{El}_i(b_N)$ gives us $\Pi\ \mathsf{El}_i(c)\ \mathsf{El}_i(d) = \Pi\ \mathsf{El}_i(c')\ \mathsf{El}_i(d')$. From this we can deduce $\mathsf{El}_i(c) = \mathsf{El}_i(c')$ and $\mathsf{El}_i(d) = \mathsf{El}_i(d')$. By the induction hypothesis we get $c = c'$ and $d = d'$, from which we conclude : $\Pi^i\ c\ d = \Pi^i\ c'\ d'$.
<u>Third case:</u> $a_N = t^i_j c$ and $b_N = t^i_k c'$. Then the equation $\mathsf{El}_i(a_N) = \mathsf{El}_i(b_N)$ gives us $\mathsf{El}_j(c) = \mathsf{El}_k(c')$. By the induction hypothesis we get $j = k$ and $c = c'$.

□

Hence, when giving models to the theory $\mathcal{T}_T$ we can interpret the decoding map as a mono $\mathsf{El}_i : \mathsf{U}_i \rightarrowtail \mathsf{Type}$. Let $\mathcal{C}$ denote the syntactic CwF of $\mathcal{T}_T$, then judgements of this theory are presheaves $\mathsf{Psh}(\mathcal{C})$, which models an *extensional* type theory. This will allow us to reason about definitional equality internally, and to derive a new judgement Small_i as such a presheaf.

Proposition 3.2.2 (Tarski-style Smallness judgement)**.** *We define*

$$\mathsf{Small}_i : \mathsf{Type} \to \Omega$$

to be the characteristic map of the mono $\mathsf{El}_i : \mathsf{U}_i \rightarrowtail \mathsf{Type}$ *in* $\mathsf{Psh}(\mathcal{C})$*. This judgement satisfy the rules of* $\mathcal{T}_F$*.*

Then we can define a coding map

$$\lceil . \rceil : \mathsf{Type} \to \mathsf{U}_i$$

which satisfies $\mathsf{El}_i(\lceil A \rceil) = A$ *for all* $A \in \mathsf{Small}_i$*.*

Proof. The characteristic map is defined by the pullback square:

$$\begin{tikzcd}
\mathsf{U}_i \arrow[r] \arrow[d, "\mathsf{El}"'] \arrow[dr, phantom, "\lrcorner", very near start] & * \arrow[d, "t"] \\
\mathsf{Type} \arrow[r, "\mathsf{Small}_i"'] & \Omega
\end{tikzcd}$$

Then, as we have coherent codes for all type connectives in U_i, we recover the smallness judgement rules of $\mathcal{T}_F$. For instance, if $A \in \mathsf{Small}_i$ and $B \in \mathsf{Small}_i$, then this means that we have two codes $a : \mathsf{U}_i$ and $b : \mathsf{U}_i$. Therefore, $\pi^i\ a\ b : \mathsf{U}_i$ and we have $\mathsf{El}_i(\pi^i\ a\ b) = \Pi(A, B)$ in Small_i.

We define $\lceil . \rceil$ as the right inverse of El_i, given by the pullback map:

$$\begin{tikzcd}
\mathsf{Type} \arrow[dr, dotted, "\lceil . \rceil"] \arrow[drr, bend left] \arrow[ddr, bend right, "\mathsf{id}"'] & & \\
& \mathsf{U}_i \arrow[r] \arrow[d, "\mathsf{El}"'] \arrow[dr, phantom, "\lrcorner", very near start] & * \arrow[d, "t"] \\
& \mathsf{Type} \arrow[r, "\mathsf{Small}_i"'] & \Omega
\end{tikzcd}$$

such that the equation $\mathsf{El}_i(\lceil . \rceil) = \mathsf{id}$ is given by the commuting triangle.

□

This gives us the following result:

Theorem 3.2.3. *The syntactic CwF of* $\mathcal{T}_T$ *is a model of the fuss-free theory* $\mathcal{T}_F$*.*

Hence, the two presentations of universe hierachies are equivalent.

3.3 A study of the different problems related to universes

When trying to build *functorial* universe hierarchies, one may encounter different problems.

Definition 3.1 (Problems related to universes)**.** We define the following problems.

- [StrictEl]: The commutation of decoding with codes for connectives.
- [StrictLift]: The commutation of lifting coercions with codes for connectives.
- [InjEl]: The injectivity of decoding.
- [InjLift]: The injectivity of lifting.

Some of the connections between those problems are already well known, and the main solution paths are listed in the following proposition.

Proposition 3.3.1. *We have the following implications.*

- *(for free)* $\underset{\textit{Schulman's local universes}}{\Longrightarrow}$ *[StrictEl]*
- *[InjLift]* $\underset{\textit{Top universe}}{\Longrightarrow}$ *[InjEl]*
- *[StrictLift]* $\underset{\textit{Top universe}}{\Longrightarrow}$ *[StrictEl]*
- *[InjLift]* $\underset{\textit{Realignment}}{\Longrightarrow}$ *[StrictLift]*

What we showed in the previous section leads to less known implications:

Proposition 3.3.2. *We proved the following.*

- *[InjEl]* $\underset{\textit{Syntax}}{\Longrightarrow}$ *[InjLift]*
- *[StrictLift, StrictEl]* $\underset{\textit{Gluing}}{\Longrightarrow}$ *[InjEl]*
- *[StrictLift, StrictEl]* $\underset{\textit{Section 2.2}}{\Longrightarrow}$ *[Fuss-Free]*

Additionally:

- *[InjEl, StrictEl]* $\underset{\textit{Syntax}}{\Longrightarrow}$ *[StrictLift]*

Thus :

$$\textit{[StrictLift,StrictEl]} \Longleftrightarrow \textit{[InjEl,StrictEl]} \Longleftrightarrow \textit{Fuss-Free}$$

This equivalence would solve a practical problem if we had a natural way to get an injective hierarchy in an arbitrary ∞-topos model, or we had a natural way to get a strict hierarchy in an arbitrary ∞-topos model. In either case, we could use the results above to interpret a very strong type theory.

We advocate for injectivity, that we believe to be the easier property in semantics. Hence, our result may allow the construction of new models which would not necessarily use properties such as realignment to get strict lifts and codes.

4 Elaboration: Embrace casts

4.1 The algorithm

We define the following elaboration rules, using a bidirectional type checking syntax.

First we have the type synthesis rules, which defines judgements $\Gamma \vdash t \overset{\mathsf{syn}}{\rightsquigarrow} t' \in T$, where everything in red is core syntax.

First we have the type checking rules, which defines judgements $\Gamma \vdash T \ni t \overset{\mathsf{syn}}{\rightsquigarrow} t'$:

To deal with unification properly, we need to introduce casts in the user syntax. The casting is defined uniformly at all types in the following way. In the surface language we have two injections of the synthesisable terms into the checkable terms:

$$\text{Checkable} := \text{Synthesisable} \mid \mathsf{cast}\ (\text{Synthesisable})$$

Hence we need two mode switch rules:

In the first case, the mode switch rule is the usual one which requires that the checking type is definitionally equal to the synthesised type. In the second case, we attempt to formulate a cast by asking a new casting judgement in the premise.

The casting judgement rules define $\Gamma \vdash \mathsf{cast}\ M : B\ \mathsf{to}\ A \rightsquigarrow M'$:

In the user syntax, the cast notation enables the user to tell the elaborator to try to use sub-typing or cummulativity instead of trying to unify types directly. Namely the casting syntax indicates the elaborator to use the second mode switch rule instead of the first one.

The requirement for this cast notation comes from the fact that unification and implicit coercion do not play well together, due to scheduling issues in the type checking. The main cause of the problem is that we cannot resolve a coercion $\gamma : \alpha \triangleleft A$ by solving $\alpha := A$ and $\gamma := 1_A$ as this is not at all general solution. Hence, even if a system has both reliable unification and implicit coercion mechanisms, then the combination of the two will invariably be unreliable.

In the case where there is a meta-variable in the casting judgement, we *postpone* the casting problem and wait until that meta-variable gets solved. If it never gets solved, the postponed problems are reported to the user.

4.2 Soundness

We can prove a soundness result for our bidirectional type checking algorithm, which ensures that we are translating to well-typed terms. First, we prove that the casting judgement does produce a term of the right type.

Theorem 4.2.1 (Soundness of Casting). *If* $\Gamma \vdash M : B$ *and* $\Gamma \vdash \mathsf{cast}\ M : B\ \mathsf{to}\ A \rightsquigarrow M'$, *then* $\Gamma \vdash M' : A$

Proof. By induction on the casting judgement. □

Theorem 4.2.2 (Soundess of Elaboration)**.** *We have the following:*

- *If* $\Gamma \vdash t \overset{\mathsf{syn}}{\rightsquigarrow} t' \in T$*, then* $\Gamma \vdash T$ *and* $\Gamma \vdash t' : T$ *i.e. a type and term elaborated from synthesis is well-typed.*
- *If* $\Gamma \vdash T \ni t \overset{\mathsf{chk}}{\rightsquigarrow} t'$*, then* $\Gamma \vdash t' : T$ *i.e. the term elaborated by checking an expression against type* T *is indeed of type* T*.*

This means that our elaboration procedure produces well-typed terms in our core language.

Proof. By mutual induction on the type synthesis and checking judgements, using theorem 4.2.1 in the case of the cast mode switch. □

Note that this result is only a safeguard and does not guarantee that our algorithm is entirely well-behaved, and one should see the elaboration algorithm as a way to define the semantics of the user syntax in terms of the core language.

For example, the user writes $\Pi(A, B) : \mathsf{U}_i$ which gets translated to $\Pi^i(A, B) : \mathsf{U}_i$, the code for a product type in the core language. In the kernel, those lifts and codes for types are implemented taking benefits from the fuss-free hierarchy.

Example 4.1.

References

[Str05] Thomas Streicher. "Universes in Toposes". In: *From Sets and Types to Topology and Analysis*. 1st ed. Oxford University Press, 2005, pp. 78–90. DOI: 10.1093/acprof:oso/9780198566519.003.0005.

[Awo24] Steve Awodey. "On Hofmann–Streicher universes". In: *Math. Struct. Comp. Sci.* 34 (2024), pp. 894–910. DOI: 10.1017/S0960129524000203.

[GSS24] Daniel Gratzer, Michael Shulman, and Jonathan Sterling. "Strict universes for Grothendieck topoi". In: (2024). DOI: 10.48550/arXiv.2202.12012.